# Autoencoders (Foundations)

Structured Notes

Encoder-Decoder Architecture, U-Net,

Autoencoders,
Denoising Autoencoders, and Reconstruction Loss Functions

# Contents

**1 Encoder-Decoder Architecture** **2**
1.1 The Encoder . . . 2
1.2 The Bottleneck (Latent Space Representation) . . . 2
1.3 The Decoder . . . 3
1.4 Applications of Encoder-Decoder Architecture . . . 3
1.5 Encoder-Decoder Architecture Diagram . . . 3

**2 U-Net Architecture** **4**
2.1 Encoder Path (Downsampling) . . . 4
2.2 Bottleneck . . . 4
2.3 Decoder Path (Upsampling) . . . 4
2.4 Skip Connections . . . 4
2.5 U-Net Architecture Summary . . . 5

**3 Autoencoders** **5**
3.1 Key Characteristics . . . 5
3.2 Autoencoder vs Encoder-Decoder Architecture . . . 5
3.3 Latent Variables and Latent Space . . . 6
3.4 Applications of Autoencoders . . . 6

**4 Denoising Autoencoders** **6**
4.1 Motivation and Concept . . . 6
4.2 Architecture of Denoising Autoencoder . . . 7
4.3 Gaussian Noise in Denoising Autoencoders . . . 7
4.4 Applications of Denoising Autoencoders . . . 7
4.5 Advantages and Limitations . . . 7

**5 Reconstruction Loss Functions** **7**
5.1 Mean Squared Error (MSE) . . . 7
5.2 Binary Cross-Entropy (BCE) . . . 8

**6 Dimensionality Reduction with Autoencoders** **8**
6.1 Data Compression . . . 8
6.2 Feature Extraction for Machine Learning . . . 9
6.3 High-Dimensional Data Visualization . . . 9

**7 Neural Inpainting** **9**

**8 Summary** **9**

# 1 Encoder-Decoder Architecture

**Encoder-Decoder Architecture**

An **encoder-decoder architecture** is a framework composed of three parts:

1. **Encoder** — Takes the input (image, speech, text) and encodes it into a compressed vector representation
2. **Bottleneck / Latent Space** — The compressed, lower-dimensional representation of the input that captures the most essential features
3. **Decoder** — Takes the compressed representation and reconstructs the desired output

This architecture is applicable to many tasks including machine translation, image captioning, semantic segmentation, and speech recognition.

## 1.1 The Encoder

The encoder is the first part of the architecture. Its role is to process the input data and extract the most important features from it through multiple layers.

- For **image data**: A CNN-based encoder is used because convolution operations capture spatial and semantic information between pixels
- For **speech/1D signal data**: A neural network (fully connected) based encoder is used since 1D data does not require spatial convolution
- The encoder gradually **reduces the dimensionality** of the input while improving abstraction
- Each layer extracts progressively finer and more meaningful details from the input
- The encoder retains only the most relevant information and discards irrelevant details

**CNN vs Neural Network for Encoding**

If an image is fed into a standard neural network, the spatial coherence between pixels is lost. For example, a smooth region of white color in an image would not be properly captured by a neural network. CNN architecture preserves this spatial information through convolution operations. Therefore, CNN is used for image-based encoding, while standard neural networks are used for 1D signal-based encoding.

## 1.2 The Bottleneck (Latent Space Representation)

The center part of the encoder-decoder architecture is the **compressed vector** or **latent representation**.

- The bottleneck is a compact representation of the input data
- It captures all essential features and patterns necessary to reconstruct the desired output
- The number of representations in the bottleneck layer is significantly smaller than the input dimensions
- The term “bottleneck” comes from the analogy of a bottle’s narrow neck — only a minimum amount of information can pass through at one time
- Ideally, the latent representation should not have significant loss of information

**Lossy vs Lossless Compression**

Compression in the bottleneck can be either **lossy** or **lossless**:

- **Lossless compression**: No information is lost; the original input can be fully reconstructed
- **Lossy compression**: Some information is lost; the original input cannot be perfectly recovered

The goal is to achieve lossless compression where the latent representation completely characterizes the input. If information is lost, performance will degrade for the target application.

## 1.3 The Decoder

The decoder is the final part of the architecture. It takes the compressed vector from the latent space and reconstructs the output.

- For image data, the decoder uses **transpose convolution** (also called deconvolution) — the opposite of the convolution process
- The decoder gradually **increases the dimensionality** back to the original input size
- The output is reconstructed in the same space as the desired target (e.g., image space for image outputs)
- The decoder retrieves detailed information corresponding to the desired target output

**Transpose Convolution (Deconvolution)**

**Transpose convolution** is the reverse operation of convolution. While convolution reduces spatial dimensions through filtering and pooling, transpose convolution increases spatial dimensions through upsampling. In an encoder-decoder architecture:

- Encoder uses convolution + pooling to downsample
- Decoder uses transpose convolution + upsampling to reconstruct the output

## 1.4 Applications of Encoder-Decoder Architecture

The encoder-decoder architecture is used when the output needs to be in the same space as the input (e.g., image-to-image), rather than just classification:

- **Image Denoising** — Input: noisy image, Output: noise-free image
- **Image Segmentation** — Input: scene image, Output: segmented regions (road, car, building, tree, sky)
- **Image Captioning** — Input: image, Output: text description of the scene
- **Medical Imaging** — Input: MRI image, Output: tumor region identification
- **Speech Recognition** — Input: speech signal, Output: speaker identification or transcription

**Encoder-Decoder vs CNN with Fully Connected Layer**

In a standard CNN architecture, the convolution layers extract features and a fully connected layer performs classification. In an encoder-decoder architecture, the fully connected classification layer is replaced by a decoder. This is necessary when the output needs to be an image (or similar spatial data) rather than a class label. The encoding process (CNN layers) is the same as feature extraction in standard CNN, but instead of classifying, the features are decoded back to image space.

## 1.5 Encoder-Decoder Architecture Diagram

The complete encoder-decoder process for image-based applications:

| Encoder | Bottleneck | Decoder |
|---|---|---|
| Input Image | | Reconstructed Output |
| Convolution Layers | Latent Space | Transpose Convolution |
| Batch Normalization | Compressed Vector | Upsampling |
| ReLU Activation | Fewer Representations | ReLU Activation |
| Max Pooling (downsample) | | Increasing Dimensions |

# 2 U-Net Architecture

**U-Net**

**U-Net** is a U-shaped encoder-decoder network that is a special case of the encoder-decoder architecture. It consists of:

- **Left side of U**: Encoder (downsampling path)
- **Bottom of U**: Bottleneck
- **Right side of U**: Decoder (upsampling path)
- **Skip connections**: Direct connections between corresponding encoder and decoder layers

The left half is a mirror image of the right half, which is why it is called U-Net.

## 2.1 Encoder Path (Downsampling)

The encoder path in U-Net consists of repeated application of:

1. Two layers of $3 \times 3$ convolution operations
2. ReLU activation function after each convolution layer
3. $2 \times 2$ max pooling for downsampling (reduces size by half)

- At each downsampling step, the spatial dimensions are halved (e.g., $568 \rightarrow 280 \rightarrow 140$)
- This reduces computational cost by decreasing the number of learnable parameters
- The encoder learns abstract-level features from the input image

## 2.2 Bottleneck

The bottleneck in U-Net:

- Connects the encoder and decoder
- Captures high-level semantic features from the encoding process
- Processes compressed high-level semantic features before passing them to the decoder

## 2.3 Decoder Path (Upsampling)

The decoder path mirrors the encoder:

1. $2 \times 2$ transpose convolution to increase the size by double (opposite of max pooling)
2. Concatenation with the corresponding feature map from the encoder via skip connection
3. Two $3 \times 3$ convolution operations

## 2.4 Skip Connections

**Skip Connections in U-Net**

**Skip connections** are direct connections between corresponding layers in the encoder and decoder. They transfer feature maps from the encoder to the decoder at the same resolution level.

**Purpose:** During downsampling in the encoder, information may be lost. Skip connections preserve this information by sharing it directly with the decoder, ensuring that fine-grained spatial details are not lost during the encoding-decoding process.

**Key Difference: U-Net vs Standard Encoder-Decoder**

The **only structural difference** between U-Net and a standard encoder-decoder architecture is the presence of skip connections. In a standard encoder-decoder, the decoder relies solely on the bottleneck representation. In U-Net, the decoder receives additional information from the encoder at each level through skip connections, which helps preserve spatial information that

might otherwise be lost during downsampling.

### 2.5 U-Net Architecture Summary

| Component | Operation | Effect |
|---|---|---|
| Encoder Conv | $3 \times 3$ convolution + ReLU | Feature extraction |
| Encoder Pooling | $2 \times 2$ max pooling | Halve spatial dimensions |
| Bottleneck | Convolution layers | Capture high-level semantics |
| Decoder Upsample | $2 \times 2$ transpose convolution | Double spatial dimensions |
| Skip Connection | Concatenation | Preserve encoder information |
| Decoder Conv | $3 \times 3$ convolution | Refine upsampled features |

## 3 Autoencoders

**Autoencoder**

An **autoencoder** is a type of neural network architecture designed to efficiently compress the input data down to its essential features and then reconstruct the **original input** from the compressed representation. Unlike the general encoder-decoder architecture where input and output can be different, in an autoencoder the input and output are typically identical.

### 3.1 Key Characteristics

- **Unsupervised learning**: Autoencoders are an unsupervised machine learning technique — no labeled training data is required. The network learns to reconstruct its own input.
- **Input $\approx$ Output**: The autoencoder attempts to produce an output that is as close to the original input as possible
- **Latent representation**: The bottleneck layer represents information as latent variables — compressed features that cannot be directly observed but can be extracted through the encoding process
- **Forced compression**: Since the latent space has fewer dimensions than the input, the autoencoder is forced to learn the most efficient compression of the data

### 3.2 Autoencoder vs Encoder-Decoder Architecture

| Aspect | Encoder-Decoder | Autoencoder |
|---|---|---|
| Learning Type | Supervised | Unsupervised |
| Input-Output Relationship | Input and output can be different (e.g., image $\rightarrow$ caption) | Input and output are typically identical |
| Framework | General framework for sequence-to-sequence or image-to-image tasks | Specific architecture for learning compressed representations |
| Applications | Machine translation, text summarization, image segmentation, speech recognition | Dimensionality reduction, feature learning, denoising, anomaly detection |
| Training Data | Requires labeled input-output pairs | Uses only input data (self-supervised) |

### 3.3 Latent Variables and Latent Space

**Latent Space**

The **latent space** (or latent variable space) refers to the compressed representation space where data is encoded in its most refined, abstract form. The concept comes from the **information bottleneck** principle (1999): the process of constructing vital information by compressing the amount of information that can traverse through the network.

Key properties of latent variables:

- Cannot be observed directly from the input
- Represent the most fundamental information in abstract form
- Extracted through the CNN or neural network encoding process
- The number of dimensions in latent space is smaller than all other layers

### 3.4 Applications of Autoencoders

Autoencoders are used for several distinct applications:

1. **Dimensionality Reduction** — Reducing high-dimensional data to a lower-dimensional representation
2. **Feature Learning** — Extracting relevant features for downstream machine learning tasks (classification, clustering, regression)
3. **Denoising** — Removing noise from corrupted data
4. **Anomaly Detection** — Identifying abnormal patterns in data distributions
5. **Image Segmentation** — Segmenting different regions in an image
6. **Neural Inpainting** — Recovering occluded or missing parts of an image
7. **Data Compression** — Encoding large datasets (images, audio, signals) into fewer representations for transmission over low-bandwidth channels
8. **Data Visualization** — Reducing high-dimensional data to 2 or 3 dimensions for better visualization

## 4 Denoising Autoencoders

**Denoising Autoencoder**

A **denoising autoencoder** is a variant of the standard autoencoder that is specifically designed to remove noise from corrupted input data and reconstruct a clean version. The autoencoder learns to differentiate between the actual signal and the noise, producing a noise-free output.

### 4.1 Motivation and Concept

Consider the scenario of satellite imagery: a satellite orbiting Mars captures images of the surface using different thematic mappers (sensors capturing images at different bands and resolutions). When multiple images are stitched together to create a complete surface image, some captured images may be corrupted by noise. The goal is to remove this noise and create a clean, usable image.

**Training Data Challenge**

To train a denoising autoencoder, a noise-free reference image is required as the target output. Since naturally corrupted images do not come with their noise-free counterparts, the typical approach is:

1. Start with a known clean (noise-free) image
2. Add synthetic noise (typically Gaussian noise) to create a corrupted version
3. Train the autoencoder with the noisy image as input and the clean image as the target

The key assumption is that the synthetic noise model closely matches the actual noise present in

real-world data.

### 4.2 Architecture of Denoising Autoencoder

1. **Encoder**: Takes the noisy/corrupted image as input. Uses CNN layers (for images) or neural network layers (for 1D signals) to extract meaningful features while being robust to the noise corruption. Produces a lower-dimensional code (latent representation).
2. **Bottleneck**: Captures the most important and meaningful information from the input data, discarding the noise. Only noise-free features are allowed to pass through to the decoder.
3. **Decoder**: Takes the noise-free latent representation and reconstructs the original clean image. The output should match the original noise-free image as closely as possible.

### 4.3 Gaussian Noise in Denoising Autoencoders

Most denoising autoencoder models use **Gaussian noise** as the corruption model. The primary reason is that Gaussian noise is:

- Mathematically well-defined and easy to implement
- A reasonable approximation for many real-world noise sources
- Computationally efficient to generate and work with

Other noise types are more difficult to model and implement, making Gaussian noise the standard choice.

### 4.4 Applications of Denoising Autoencoders

- **Feature Learning** — Learning robust latent representations from corrupted data
- **Data Pre-processing** — Cleaning data before feeding it into other machine learning models (e.g., identifying minerals on a moon surface from noisy satellite images)
- **Image Processing in Computer Vision** — Image denoising, inpainting, super-resolution
- **Anomaly Detection** — Identifying abnormalities in noisy data distributions

### 4.5 Advantages and Limitations

| Advantages | Limitations |
|---|---|
| Effective at learning latent representations from corrupted data | Requires construction of a preliminary stochastic noise mapping |
| Creates robust representations that allow complete recovery of the original image | The synthetic noise model may not match the actual noise in real-world data |
| Some information loss is acceptable and does not impair performance | Performance degrades if the noise model is significantly different from actual corruption |

## 5 Reconstruction Loss Functions

Reconstruction loss functions measure the difference between the original input and the reconstructed output. The primary objective of autoencoder training is to minimize this reconstruction loss. Two main types of reconstruction loss functions are used:

### 5.1 Mean Squared Error (MSE)

**Mean Squared Error (MSE)** for image reconstruction:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{M}\sum_{j=1}^{M}\sum_{i=1}^{d}(x_i^{(j)} - \hat{x}_i^{(j)})^2$$

Where:

- $x_i^{(j)}$ is the $i$-th feature of the $j$-th original input sample
- $\hat{x}_i^{(j)}$ is the $i$-th feature of the $j$-th reconstructed output
- $d$ is the number of feature dimensions (total pixels for images)
- $M$ is the mini-batch size (number of images used for averaging)

The loss is computed by:

1. Computing the squared difference between original and predicted values for each feature
2. Summing over all feature dimensions $d$
3. Averaging over all $M$ samples in the mini-batch

### 5.2 Binary Cross-Entropy (BCE)

**Binary Cross-Entropy (BCE)** for reconstruction:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{M}\sum_{j=1}^{M}\sum_{i=1}^{d}[x_i \log(\hat{x}_i) + (1 - x_i)\log(1 - \hat{x}_i)]$$

Where:

- $x_i$ is the actual output value for the $i$-th pixel (0 or 1 for binary)
- $\hat{x}_i$ is the predicted output for the $i$-th pixel
- This function is primarily applicable for binary classification tasks (object vs background)

Binary cross-entropy is used when the reconstruction task involves binary outputs, such as identifying whether each pixel belongs to an object (1) or the background (0).

**Choosing the Loss Function**

- Use **MSE** when the output values are continuous (e.g., pixel intensity values in grayscale or RGB images)
- Use **Binary Cross-Entropy** when the output is binary (e.g., segmentation masks where each pixel is either object or background)

The training process iteratively updates the network weights through backpropagation to minimize the chosen reconstruction loss function.

## 6 Dimensionality Reduction with Autoencoders

Autoencoders serve as a powerful tool for dimensionality reduction, converting high-dimensional input data into low-dimensional latent representations.

### 6.1 Data Compression

- Large datasets (images, audio, signals) can be encoded into fewer number of representations
- Useful for transmitting data over low-bandwidth channels
- The compressed representation retains the essential information needed for the target task

### 6.2 Feature Extraction for Machine Learning

When dealing with high-dimensional feature spaces, autoencoders can extract only the most relevant features:

- The encoder learns which features are most important for representing the input
- Irrelevant features are discarded during the compression process
- The reduced feature set can be used as input for downstream tasks: classification, clustering, or regression
- This serves as a pre-processing tool that improves the efficiency and performance of subsequent machine learning models

### 6.3 High-Dimensional Data Visualization

- High-dimensional data that cannot be directly visualized can be reduced to 2 or 3 dimensions
- The reduced representation enables better visualization and understanding of data patterns
- Autoencoders provide a non-linear alternative to linear dimensionality reduction methods like PCA

## 7 Neural Inpainting

**Neural Inpainting**

**Neural inpainting** is the process of recovering occluded or missing parts of an image using an autoencoder. When a portion of an image is hidden or obscured by another object in the scene, the autoencoder can be trained to reconstruct the missing content based on similar scenes from previous instances.

The process works as follows:

1. The autoencoder is trained on complete images of similar scenes
2. When presented with an image where part of the content is occluded, the encoder compresses the available information
3. The latent representation captures the essential structure of the scene
4. The decoder reconstructs the full image, including the previously hidden regions
5. The reconstructed content is based on patterns learned from the training data

## 8 Summary

- The **encoder-decoder architecture** is a general framework composed of an encoder, bottleneck (latent space), and decoder — used when output needs to be in the same space as input rather than just classification
- For image data, CNN-based encoders and transpose convolution-based decoders are used; for 1D data, neural network-based architectures are used
- **U-Net** is a special case of encoder-decoder architecture with skip connections between encoder and decoder layers, which preserve spatial information that may be lost during downsampling
- An **autoencoder** is an unsupervised variant where the output is identical to the input, used for dimensionality reduction, feature learning, denoising, and anomaly detection
- **Denoising autoencoders** are trained by adding synthetic noise (typically Gaussian) to clean images and learning to reconstruct the noise-free version
- Two primary reconstruction loss functions are used: **Mean Squared Error** (for continuous outputs) and **Binary Cross-Entropy** (for binary outputs)
- Autoencoders serve as effective tools for **dimensionality reduction**, **data compression**, **feature extraction**, and **neural inpainting**
- The training process uses backpropagation to iteratively minimize the reconstruction loss